

Digital Professor: Interaction Methods

Sep. 18th Update



Jonathan Ma

Prototype Review

What got done

A working tutoring experience

- ▶ Browser-based course workspace
- ▶ Syllabus and course-material uploads
- ▶ Course-grounded tutoring chat
- ▶ Source citations and learning check-ins

Multiple ways to interact

- ▶ Typed questions, equations, and code
- ▶ Documents and images
- ▶ Handwriting with a sketchpad or stylus
- ▶ Recorded voice converted to editable text

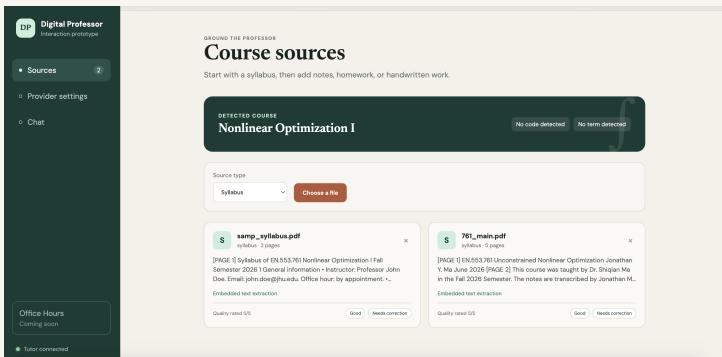
Outcome: an end-to-end prototype for testing how students might naturally work with a Digital Professor.

The high-level student experience



- ▶ The system builds a course-aware workspace from student-provided material.
- ▶ Students choose the interaction method that fits the problem they are solving.
- ▶ The tutor explains, cites its course sources, and checks understanding.

Demo: course setup and source material



What worked

- ▶ Course material was uploaded and made available to the tutor
- ▶ The course name was identified
- ▶ Students can review the extracted sources

What needs work

- ▶ Some metadata was missed
- ▶ Extracted information needs an edit-and-confirm step

Demo: tutoring interaction

Digital Professor
Nonlinear Optimization I

Why does normalizing the exponentials so that both sets of weights sum to 1 make the AM-GM bound useful?

Assuming you mean the AM-GM proof of convexity of log-sum-exp, the key is normalization makes the summed AM-GM upper bound exactly 1, while keeping the original scales outside the sum.

Let

$$A = \sum_{i=1}^n e^{a_i}, \quad B = \sum_{i=1}^n e^{b_i}$$

and normalize

$$p_i = \frac{e^{a_i}}{A}, \quad q_i = \frac{e^{b_i}}{B}$$

Then $\sum_{i=1}^n p_i = \sum_{i=1}^n q_i = 1$.

1. AM-GM now gives a useful constant bound

For $0 \leq t \leq 1$, weighted AM-GM gives, for each i ,

$$p_i^t q_i^{1-t} \leq tp_i + (1-t)q_i$$

What worked

- ▶ Course-specific worked explanation
- ▶ Readable mathematical notation
- ▶ Assumptions and comprehension check

Digital Professor
Nonlinear Optimization I

2. Put the original scales back

Since $e^{a_i} = A p_i$ and $e^{b_i} = B q_i$,

$$\sum_{i=1}^n t a_i + (1-t) b_i = A^t B^{1-t} \sum_{i=1}^n p_i^t q_i^{1-t}$$

Taking together gives precisely the convexity inequality:

$$\log \sum_{i=1}^n t e^{a_i} + (1-t) \log \sum_{i=1}^n e^{b_i} \leq t \log \sum_{i=1}^n e^{a_i} + (1-t) \log \sum_{i=1}^n e^{b_i}$$

Why not apply AM-GM directly? That would give an upper bound of $tA + (1-t)B$, rather than the needed $A^t B^{1-t}$. The arithmetic bound is generally larger, so it is too weak to establish this conclusion. Thus normalization is not required to apply AM-GM—it makes AM-GM yield the right bound.

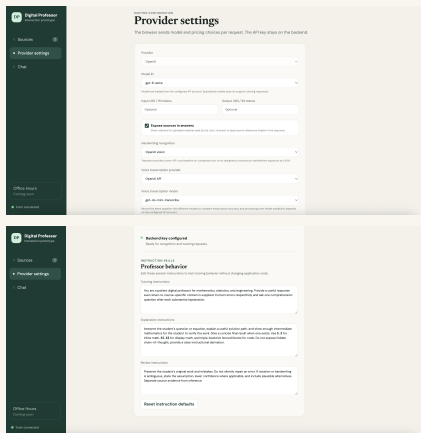
CHECK YOUR UNDERSTANDING

If the two normalized sums were instead both equal to a constant c , what upper bound would summing AM-GM give?

What needs work

- ▶ Responses can be too long
- ▶ Latency is noticeable
- ▶ Citation quality needs evaluation

Demo: the experience is configurable



- ▶ Tutoring behavior can be adjusted without changing code
- ▶ Source visibility and interaction options can be changed for an experiment
- ▶ This gives us a practical way to tune the experience during evaluation

Design issue: the settings need to become simpler before student use.

Interaction methods now available

Course material	Live writing	Voice
Upload syllabi, notes, homework, images, and handwritten work so the tutor can answer in context.	Draw a question or equation with a mouse, finger, or stylus and submit it from the chat.	Record a question, review the transcription, correct it if needed, and send it to the tutor.

The prototype records quality feedback, correction effort, response time, and estimated cost so these methods can be compared.

What we learned from the first demo

Promising

- ▶ The complete student flow is understandable
- ▶ Multiple input methods fit different learning tasks
- ▶ Mathematical output is readable
- ▶ Course grounding makes the tutor feel more relevant

Unresolved

- ▶ We have not shown that the tutoring is consistently correct or helpful
- ▶ Answers can be too detailed for the student's need
- ▶ Recognition and citations still fail on ambiguous inputs
- ▶ Latency can interrupt the tutoring rhythm

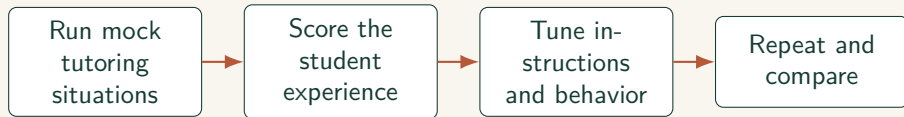
Next milestone: mock tutoring situations

Tune and evaluate the tutoring experience

Build a repeatable set of realistic student–professor interactions, score the prototype, adjust its tutoring behavior, and run the same situations again.

- ▶ A student asks for a conceptual explanation at the wrong level of detail
- ▶ A student has a misconception and needs guided correction
- ▶ A student requests help on homework without being given the answer
- ▶ A student submits flawed work and asks for review
- ▶ A student asks a question that must be answered from course material
- ▶ A handwritten or spoken equation is ambiguous and requires clarification

How the tuning and evaluation cycle works



Quality criteria

- ▶ Correctness and source grounding
- ▶ Pedagogical usefulness and appropriate hints
- ▶ Recognition and recovery from ambiguity

Practical criteria

- ▶ Student correction effort
- ▶ Response time and interaction flow
- ▶ Cost and consistency across repeated trials

Blockers and roadmap

Current blockers

- ▶ No agreed scoring rubric or ground-truth tutoring examples yet
- ▶ Handwriting and spoken mathematics remain ambiguous
- ▶ Response time can be too slow for natural dialogue
- ▶ The prototype is local and does not retain student work

Roadmap after evaluation

1. Improve course grounding and citations
2. Add persistent course workspaces
3. Pilot a secure public web application
4. Prototype real-time Office Hours
5. Add specialized agents only if evaluation shows a need

Decision we want from the next milestone

Is the prototype acting like a useful tutor?

The next milestone should tell us which tutoring behaviors work, where students must correct the system, and whether the experience is accurate, grounded, responsive, and affordable enough to develop further.

- ▶ Keep the interaction methods that improve student success
- ▶ Tune explanations, hints, review behavior, and comprehension checks
- ▶ Use evidence to choose models and decide whether more architecture is necessary

Questions and prototype walkthrough